Ai_Branch - Verslag en ontwikkelstappen

> Overzicht van het AI-prototype voor O&P Rijk, inclusief de projectuitleg en de ontwikkelstappen uit de map commits.

code\README.md

````
# Nudge naar Dienst

Een starter voor een AI-gestuurde nudgebot. De bot stelt eerst vier intakevragen, daarna twee korte vervolgvragen en geeft vervolgens een persoonlijke, concrete nudge richting een passende dienst.

## Starten

De demo gebruikt een lokale Python-server en Ollama. Installeer Ollama en haal het gekozen model op:

```powershell
ollama pull qwen3:4b
```

Start daarna de demo vanuit de map `JS Provocatype`:

```powershell
python server.py
```

Open `http://127.0.0.1:8000`. De server bindt alleen aan je eigen computer. Zorg dat Ollama actief is.

## Projectstructuur

```text
src/
  data/          Vragen en dienstencatalogus
  AI/            Regelgebaseerde dienstkeuze en Ollama-client
  app.js         Schermflow en invoerverwerking
  index.html     Startpagina
  styles.css     Vormgeving
docs/            Architectuur en vervolgstappen
tests/           Tests voor de Ollama-server
server.py        Lokale webserver en Ollama-proxy
```
````

````
## Hoe de AI werkt

De vaste regels in `src/AI/nudgeEngine.js` kiezen drie diensten uit de O&P Rijk-catalogus
(2026) en vinken de eerste vooraf aan. Als een open antwoord onduidelijk is, maakt het
ingestelde Ollama-model via `/api/follow-up` een contextuele vervolgvraag én een toelichting
die als vraagbeschrijving in het formulier verschijnt; er verschijnen maximaal twee extra
vragen per formulier, ook als de melding na twee onbruikbare antwoorden al wordt getoond. Dan
waarschuwt de demo dat de uiteindelijke aanbeveling algemener of minder zeker kan zijn. Een
leeg antwoord op een vaste vraag blijft verplicht; een extra vraag mag worden overgeslagen.
Qwen3 4B schrijft de nudgetekst en een korte reden als gestructureerde JSON (`format`-schema in
`server.py`). Na de keuze toont het eindformulier per AI-stap de gebruikte model-ID, diens
korte reden, vraagbeschrijving en uitvoer. De dienstkeuze zelf komt van vaste regels, niet van
een LLM. De browser stuurt antwoorden naar `server.py`, die lokaal doorstuurt naar Ollama; er
is geen API-sleutel nodig. Rechtsboven staat een gezicht dat blij, neutraal, verward of
teleurgesteld kijkt (`src/AI/answerJudge.js`). Klik erop voor het logboek met de echte stappen
van de app, niet de verborgen redeneertekst van een model. De demo is alleen bedoeld als
gesimuleerd experiment, nooit voor echte gebruikers.

Dit is geen fine-tuning: Qwen3 is niet opnieuw getraind. De demo geeft het bestaande model een
beperkte opdracht en JSON-schema. Om de kwaliteit te verbeteren, test je verschillende
voorbeeldantwoorden en pas je de opdracht aan. Fine-tuning is pas te overwegen als je een
grotere, gecontroleerde verzameling goede vraag/antwoord-voorbeelden hebt.

Als Ollama niet bereikbaar is of het model ontbreekt, toont de demo een fout en kun je zelf
kiezen voor de standaardnudge zonder AI.

Zie `docs/architecture.md` voor de voorgestelde volgende stappen.

## Tests

```powershell
python -m unittest discover -s tests
```
````

commits\001-omgeving-gemaakt.md

```
# Commit 1: Omgeving gemaakt

**Datum:** 8 oktober 2026

## Wat is gedaan

Basis-omgeving voor de AI website opgezet. Dit is een voorbereiding, zonder AI functionaliteit.

## Mappen gemaakt

- `src/` (Main folder voor frontend code)
- `test/` (Main folder voor tests)
- `docs/` (Documentatie)
```

```
## Files gemaakt

### Root level
- `.gitignore` - Git configuratie
- `README.md` - Project beschrijving

### src/ (Frontend code)
- `index.html` - Homepage
- `app.js` - Basis JavaScript app
- `styles.css` - CSS styling

### src/AI/
- `nudgeEngine.js` - Basis voor nudge logica
- `aiProvider.js` - Basis voor AI communicatie

### src/data/
- `questions.js` - Vragenlijst structuur

### test/
- `test.html` - Test pagina
- `main.py` - Python test script (leeg)

## Waarom

Dit is de basis waarmee alles begint. De omgeving is klaar maar nog zonder AI.

## Volgende stap

Toevoegen van echte AI-functionaliteit en validatie.
```

commits\002-core-ai-engine.md

```
# Commit 2: Core AI Engine Geinitialiseerd

**Datum:** 21 oktober 2026

## Wat is gedaan

Volledige AI-gestuurde nudge engine gebouwd. Dit is het hart van het project.

## Mappen gemaakt

- `JS Provocatype/` (Folder voor JavaScript versie van de app)
- `Python Engine/` (Folder voor Python backend)
- `JS Provocatype/docs/` (Documentatie voor JS versie)
- `Python Engine/backend/` (Backend code)
- `Python Engine/frontend/` (Frontend assets)
- `Python Engine/tests/` (Tests)
```

```
## Folders/Files volledig gemaakt

### JS Provocatype/ (JavaScript versie)
- `README.md` - Uitleg hoe te gebruiken
- `server.py` - Python webserver voor demo
- `src/AI/nudgeEngine.js` - Regelgebaseerde dienst matching (10 O&P Rijk diensten)
- `src/AI/aiProvider.js` - Communicatie met Ollama AI
- `src/app.js` - Frontend logica
- `src/data/questions.js` - 6 intakevragen (situatie, doel, barriere, ondersteuning, timing,
contact)
- `src/index.html` - Homepage met vragenformulier
- `src/styles.css` - Styling
- `tests/test_server.py` - Server tests

### Python Engine/ (Organisatie & workflows)
- `backend/app/engine/catalogue.py` - Dienstencatalogus
- `backend/app/engine/policy.py` - Bedrijfsregels
- `backend/app/engine/profiler.py` - Gebruiker profiel matching
- `backend/app/main.py` - Flask API server
- `backend/app/storage.py` - Data opslag
- `frontend/` - HTML/JS/CSS voor Python versie
- `tests/` - Test suite
- `requirements.txt` - Python dependencies
- `run_windows.bat` / `run_unix.sh` - Start scripts

### Documentatie
- `docs/architecture.md` - Architectuur uitleg
- `docs/DEVELOPMENT_PLAN.md` - Plan voor volgende stappen
- `PROJECT_MAP.md` - Overview van al files
- `Verslag_model.md` - Uitleg over het AI model

## Waarom

Het project spitstte zich op in 2 versies:
1. **JS Provocatype** - Simpele demo met regels + Ollama AI
2. **Python Engine** - Complexere backend met organisatie hierarchie

Dit is de eerste grote milestone waar alles werkend is.

## Volgende stap

Toevoegen van geavanceerde features zoals antwoord beoordeling en vervolgvragen.
```

commits\003-antwoord-beoordeling.md

```
# Commit 3: AI Antwoord Beoordeling & Gezicht Interactie

**Datum:** 27 oktober 2026

## Wat is gedaan
```

````
Gebruiker feedback systeem toegevoegd. De app reageert nu op antwoorden en geeft visuele
feedback.

## Files gemaakt/gewijzigd

### JS Provocatype/
- `src/AI/answerJudge.js` - NIEUW: Beoordeelt antwoorden (blij, verward, verdrietig, neutraal)
- `src/face.js` - NIEUW: Gezicht dat reageert met emoties
- `src/data/questions.js` - Verbeterd met meer vraag details
- `server.py` - Verbeterd met validatie
- `src/app.js` - Aangepast voor gezicht integratie
- `src/index.html` - Nieuw gezicht element
- `src/styles.css` - Styling voor gezicht
- `README.md` - Documentatie updated
- `docs/architecture.md` - Updated
- `tests/test_server.py` - Tests toegevoegd

### Documentatie
- `PROJECT_MAP.md` - NIEUW: Volledige file overview
- `Verslag_model.md` - NIEUW: Uitleg over model keuzes

## Waarom

De app voelde statisch. Door een gezicht toe te voegen dat reageert op input maakt het:
- Meer interactief
- User-friendly
- Duidelijk wat de app aan het doen is

Antwoord beoordeling helpt om:
- Slechte antwoorden op te sporen (te kort, irrelevant)
- Gebruiker stimuleren beter antwoord te geven
- Kwaliteit van data verbeteren

## Functie van antwordJudge.js

```javascript
judgeAnswer(question, value)
- Input: vraag ID en antwoord tekst
- Output: { mood: "happy"/"sad"/"confused"/"neutral", why: "reden" }
- Controleert: lengte, woorden, unieke letters
- Checkt tegen trefwoorden
```

## Volgende stap

Vervolgvragen toevoegen als gebruiker antwoord niet goed genoeg is.
````

commits\004-kwaliteit-vervolgvragen.md

Commit 4: AI Interactie Verbeterd - Kwaliteit & Vervolgvragen

Datum: 27 oktober 2026

Wat is gedaan

Kwaliteitsbepaling toegevoegd. App vraagt nu automatisch vervolgvragen als antwoord niet goed genoeg is.

Files gewijzigd

JS Provocatype/
- `server.py` - Verbeterd validatie voor vervolgvragen
- `src/AI/aiProvider.js` - Nieuwe endpoint voor /api/follow-up
- `src/app.js` - Integratie vervolgvragen in flow
- `src/index.html` - Dynamische vraag rendering
- `src/styles.css` - Styling voor vervolgvragen
- `README.md` - Updated docs
- `docs/architecture.md` - Updated flow
- `tests/test_server.py` - Nieuwe tests

Waarom

Met alleen hoofdvragen krijgen we soms:
- Te korte antwoorden ("Ja" / "Nee")
- Irrelevante antwoorden
- Niet genoeg informatie om goed advies te geven

Vervolgvragen zorgen voor:
- Diepere, betere antwoorden
- Meer context
- Beter AI-advies

Hoe het werkt

1. Gebruiker geeft antwoord
2. `judgeAnswer()` beoordeelt of het goed genoeg is
3. Als slecht: roep `/api/follow-up` aan
4. Ollama genereert automatisch vervolgvraag
5. Gebruiker beantwoordt vervolgvraag
6. Antwoorden gecombineerd voor aanbeveling

Limiet

Maximaal 2 vervolgvragen per formulier (anders duurt het te lang).

Volgende stap

AI-aanbeveling voor diensten (niet alleen regelgebaseerd matching).

commits\005-vervolgvragen-beschrijving.md

````
# Commit 5: Vervolgvragen Beschrijving Veld

**Datum:** 27 oktober 2026

## Wat is gedaan

Vervolgvragen hebben nu een beschrijving/toelichting. Dit helpt gebruiker beter antwoorden.

## Files gewijzigd

### JS Provocatype/
- `server.py` - Description field toegevoegd aan /api/follow-up response
- `src/AI/aiProvider.js` - Description nu uit API gehaald
- `src/app.js` - Description tonen in UI
- `src/face.js` - Integratie met face logging
- `README.md` - Updated
- `docs/architecture.md` - Updated
- `tests/test_server.py` - Nieuwe tests

## Waarom

Soms is een vervolgvraag onduidelijk. Voorbeeld:

**SLECHT:**
```
Vervolgvraag: Kunt u gespecificeerd de barriere aangeven?
(gebruiker is verward)
```

**GOED:**
```
Vervolgvraag: Kunt u gespecificeerd de barriere aangeven?
Beschrijving: We willen weten WELKE dingen je werkdruk veroorzaken.
              Is het veel werk? Te lastige taken? Slecht team?
(gebruiker begrijpt en geeft beter antwoord)
```

## Response format

```json
{
  "question": "Kun je concreter zijn?",
  "description": "We willen weten wat je precies bedoelt",
  "reason": "Antwoord was te algemeen",
  "model": "qwen3:4b",
  "adjusted": false
}
```

## Volgende stap
````

```
AI-genereerde dienstkeuze (fallback op regelgebaseerd).
```

commits\006-ai-nudging-profiel.md

```
# Commit 6: AI-Nudging & Profielpagina

**Datum:** 6 november 2026

## Wat is gedaan

Volledige AI-aanbeveling systeem + gebruikersprofielpagina.

## Mappen/Files gemaakt

### JS Provocatype/
- `server.py` - Uitgebreid met /api/nudge endpoint
- `src/AI/aiProvider.js` - Nieuwe functies voor AI communicatie
- `src/AI/nudgeEngine.js` - Regelgebaseerde fallback
- `src/app.js` - Flow met AI-nudge aanbeveling
- `tests/test_server.py` - Uitgebreide tests

### Python Engine/
- `backend/app/main.py` - Nieuwe profielpagina API
- `frontend/profile.html` - NIEUW: Profielpagina
- `frontend/profile.css` - NIEUW: Styling profielpagina
- `frontend/profile.js` - NIEUW: JavaScript logica

## Hoe AI-Nudging werkt

1. Gebruiker beantwoordt 6 vragen + mogelijk 2 vervolgvragen
2. Frontend stuurt naar `/api/nudge`
3. Server stuurt naar Ollama met:
   - Alle antwoorden
   - Regelgebaseerde aanbeveling (fallback)
   - Instructies (JSON format!)
4. Ollama genereert:
   - `message`: Persoonlijke nudge tekst
   - `reason`: Korte uitleg waarom deze dienst
5. Frontend toont nudge met opties

## Fallback mechanisme

ALS Ollama crash/niet reageerd:
- Gebruik regelgebaseerde aanbeveling
- Toon waarschuwing "beperkte informatie"
- Gebruiker kan nog kiezen

## Profielpagina
```

```
- Toon alle sessies
- Statistieken (hoeveel vragen gesteld, etc)
- Welke diensten aanbevolen
- Trend over tijd

## Volgende stap

Verbeteringen op basis van gebruikersfeedback.
```

commits\README.md

```
# Commits - Wat ik heb gemaakt in deze branch

Deze folder bevat een gestructureerd overzicht van elk commit/milestone in de Ai_branch.

## Overzicht

| # | Commit | Datum | Wat gedaan |
|---|--------|-------|-----------|
| 1 | 001 | 8 okt | Omgeving & basis code |
| 2 | 002 | 21 okt | Core AI engine (2 versies) |
| 3 | 003 | 27 okt | Antwoord beoordeling & gezicht |
| 4 | 004 | 27 okt | Kwaliteit check & vervolgvragen |
| 5 | 005 | 27 okt | Vervolgvragen beschrijving |
| 6 | 006 | 6 nov | AI-nudging & profielpagina |

## Hoe te lezen

Elk bestand (001-006) bevat:
- **Wat is gedaan**: Korte uitleg
- **Mappen gemaakt**: Welke folders zijn nieuw
- **Files gemaakt/gewijzigd**: Welke files veranderd
- **Waarom**: Reden van de keuze
- **Volgende stap**: Wat daarna kwam

## De twee versies

### JS Provocatype (Gebruikt voor demo)
- Eenvoudige JavaScript frontend
- Python webserver (Flask)
- Ollama AI backend
- 10 O&P Rijk diensten
- Regelgebaseerde matching + AI fallback

**Start:** `python server.py` -> http://127.0.0.1:8000

### Python Engine (Meer complex)
- Python Flask backend
- HTML/CSS/JS frontend
- Organisatie hierarchie
```

```
- Workflows & policies
- User profiling

## Totaal bereikt

- **6 commits** met stijgende complexiteit
- **2 werkende versies** van de app
- **AI-integratie** met Ollama
- **Regelgebaseerde matching** (10 diensten)
- **Vervolgvragen** op basis van kwaliteit
- **Profielpagina** voor analyse

## Volgende stap

De app is nu klaar voor:
1. Testing met echte gebruikers
2. Verbetering van AI prompts
3. Toevoegen meer diensten
4. Fine-tuning van regelgebaseerde matching
```

README.md

```
# Ai_Branch - Compleet Project

Dit is het eindresultaat van de Ai_branch. Een werkende AI-gestuurde nudge applicatie voor O&P
Rijk diensten.

## Structuur

```
Ai_Branch/
├── code/              -> Alle werkende code
│   ├── server.py      -> Python webserver
│   ├── src/           -> Frontend (HTML/JS/CSS)
│   ├── tests/         -> Tests
│   └── [Python files] -> Models, validators, etc
│
└── commits/           -> Beschrijving wat je hebt gemaakt
    ├── 001-omgeving-gemaakt.md
    ├── 002-core-ai-engine.md
    ├── 003-antwoord-beoordeling.md
    ├── 004-kwaliteit-vervolgvragen.md
    ├── 005-vervolgvragen-beschrijving.md
    ├── 006-ai-nudging-profiel.md
    └── README.md      -> Overzicht van alle commits
```

Hoe te starten

Vereisten
```

````
1. **Python 3.9+**
2. **Ollama** (download van https://ollama.ai)
3. **Qwen model** voor Ollama:
   ```bash
   ollama pull qwen3:4b
   ```

### Stap 1: Ollama starten

```bash
ollama serve
```

Dit start Ollama op `http://127.0.0.1:11434`

### Stap 2: De app starten

Ga naar de `code/` folder:

```bash
cd code
python server.py
```

De server start op: `http://127.0.0.1:8000`

### Stap 3: Browser openen

Ga naar: http://127.0.0.1:8000

Je ziet nu:
- Formulier met 6 vragen
- Gezicht dat reageert
- Mogelijke vervolgvragen
- Finale nudge aanbeveling

## Wat je kunt doen

1. **Vragen beantwoorden** -> App stuurt naar Ollama
2. **AI genereert vervolgvragen** -> Als antwoord niet goed genoeg
3. **AI geeft nudge** -> Persoonlijk advies naar dienst
4. **Fallback** -> Als Ollama crash: regelgebaseerde aanbeveling
5. **Logs zien** -> Klik op gezicht rechtsboven

## Hoe het werkt

### Stap-voor-stap

1. **Frontend verzamelt antwoorden** (6 vragen)
2. **Backend checkt kwaliteit** via answerJudge
3. **Slecht antwoord?** -> Ollama genereert vervolgvraag
4. **Antwoord goed?** -> Naar AI-nudge
5. **AI-nudge** -> Ollama genereert persoonlijke bericht
````

6. **Fallback** -> Regelgebaseerde matching als AI faalt

Code flow

```python
# 1. Inkomend antwoord
POST /api/nudge
{
  "answers": {
    "situation": "Ik ben gestrest",
    "goal": "Beter werkplezier",
    ...
  }
}

# 2. Server stuurt naar Ollama
# 3. Ollama genereert nudge
# 4. Response terug naar frontend
{
  "message": "Bedrijfszorg kan helpen met stress...",
  "reason": "Je noemde 'stress' en 'werkdruk'",
  ...
}
```

De commits (lees dit voor details!)

Lees `commits/README.md` voor:
- Wat in ELKE commit gedaan is
- Welke folders/files gemaakt
- Waarom keuzes gemaakt
- Hoe features werken

Foutmeldingen?

"Ollama is niet bereikbaar"
- Zorg dat `ollama serve` draait
- Check: http://127.0.0.1:11434

"Model niet gevonden"
- Voer uit: `ollama pull qwen3:4b`

"Python errors"
- Check requirements: `pip install flask urllib3`

Volgende stappen

Wat je nog kunt doen:
1. **Meer diensten toevoegen** -> service_catalogue.py
2. **AI-prompts verbeteren** -> server.py (PROMPT_* variabelen)
3. **More tests** -> tests/test_server.py
4. **Deployment** -> Docker container
5. **Fine-tuning** -> Ollama model tunen met betere prompts

```
## Vragen?

Lees:
- `code/README.md` -> Technische details
- `commits/README.md` -> Wat je hebt gemaakt
- `commits/006-ai-nudging-profiel.md` -> Meeste advanced features
```